# Template for Enrichment task 1: Programming the Hill cipher

## Question 1: Hill cipher

In [1]:
```
# Hill Function 1
def hillFunction1(ciphertext, s):
    ciphertext = ciphertext.upper().replace(" ", "")

    if s < 2:
        raise ValueError("Matrix size must be at least 2")

    if len(ciphertext) % s != 0:
        raise ValueError("Ciphertext length must be divisible by the matrix
size")

    numbers = []

    for letter in ciphertext:
        numbers.append(ord(letter) - ord('A'))

    blocks = []

    for i in range(0, len(numbers), s):
        blocks.append(numbers[i:i+s])

    return blocks
```

In [2]:
```
# Hill Function 1 testing

encoded1 = hillFunction1("YIFZMA", 2)

print(encoded1)
```

Out[2]:
```
[[24, 8], [5, 25], [12, 0]]
```

In [3]:
```
# Hill Function 2
def hillFunction2(encoded_text, encoding_matrix):

    # Work modulo 26
    A = Matrix(Integers(26), encoding_matrix)

    # Find inverse of the encryption matrix
    A_inverse = A.inverse()

    decrypted_blocks = []

    for block in encoded_text:

        # Convert block into a vector
        cipher_vector = vector(Integers(26), block)
```

```
        # Decrypt using the inverse matrix
        plain_vector = cipher_vector * A_inverse

        # Convert numbers back into letters
        letters = ""

        for number in plain_vector:
            letters += chr(int(number) + ord('A'))

        decrypted_blocks.append(letters)

    return decrypted_blocks
```

In [4]:

```
# Exercise 3.3.1

ciphertext1 = "YIFZMA"

matrix1 = [
    [9, 13],
    [2, 3]
]

encoded1 = hillFunction1(ciphertext1, 2)

print("Function 1 output:")
print(encoded1)

decrypted1 = hillFunction2(encoded1, matrix1)

print("Function 2 output:")
print(decrypted1)
```

Out[4]:

```
Function 1 output:
[[24, 8], [5, 25], [12, 0]]
Function 2 output:
['EU', 'RE', 'KA']
```

In [5]:

```
# Exercise 3.3.2

ciphertext2 = "GEZXDS"

matrix2 = [
    [12, 3],
    [11, 2]
]

encoded2 = hillFunction1(ciphertext2, 2)

print("Function 1 output:")
print(encoded2)

decrypted2 = hillFunction2(encoded2, matrix2)

print("Function 2 output:")
print(decrypted2)
```

Out[5]:

```
Function 1 output:
[[6, 4], [25, 23], [3, 18]]
```

```
Function 2 output:
['SO', 'LV', 'ED']
```

In [6]:

```sage
# Part (b) - Encrypt first name

name = "HIRA"

name_matrix = [
    [3, 3],
    [2, 5]
]

A = Matrix(Integers(26), name_matrix)

# Convert name into number blocks
name_blocks = hillFunction1(name, 2)

print("Original name blocks:")
print(name_blocks)

cipher_blocks = []

for block in name_blocks:
    plain_vector = vector(Integers(26), block)
    cipher_vector = plain_vector * A

    cipher_blocks.append(
        [int(number) for number in cipher_vector]
    )

print("Encrypted number blocks:")
print(cipher_blocks)

# Convert encrypted numbers back into letters
ciphertext_name = ""

for block in cipher_blocks:
    for number in block:
        ciphertext_name += chr(number + ord('A'))

print("Ciphertext:")
print(ciphertext_name)
```

Out[6]:

```
Original name blocks:
[[7, 8], [17, 0]]
Encrypted number blocks:
[[11, 9], [25, 25]]
Ciphertext:
LJZZ
```

In [10]:

```sage
# Test Function 1 using encrypted first name

encoded_name = hillFunction1("LJZZ", 2)

print(encoded_name)
```

Out[10]:

```
[[11, 9], [25, 25]]
```

In [8]:

```sage
# Test Function 2 using encrypted first name
```

```
decrypted_name = hillFunction2(
    encoded_name,
    [[3, 3], [2, 5]]
)

print(decrypted_name)
```

Out[8]:
```
['HI', 'RA']
```